AI Model Testing & Scenario-Based Evaluation Harness (Safe Pre-Production)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Outcome . . . 4

Approach . . . 4

Why it is safe . . . 4

Solution Overview 4

What this is . . . 4

How it works . . . 5

Target model / data . . . 5

Benefits . . . 5

Limitations . . . 6

The deployment model . . . 6

Requirements 7

R1. A model exposed through a callable "Model Under Test" contract . . . 7

R2. Ground truth / expected behavior for each scenario . . . 7

R3. Agreed evaluation metrics and pass/fail thresholds . . . 8

R4. Synthetic/twin test data mapped to the scenarios . . . 8

R5. A run/results store and a baseline location . . . 8

R6. Determinism / reproducibility controls for the run . . . 8

Global Requirements — Synthetic Data & AI Test-Enablement Solutions 9

1. A Python data/engineering environment . . . 9

2. A target clinical data model / standard . . . 9

3. An isolated non-production test environment . . . 10

4. Privacy, governance & compliance stakeholders and policy . . . 10

5. AI/QA use cases and a pilot scope . . . 10

6. Reference/real-data access for fidelity (read-only, minimized) . . . 10

Implementation 11

Stage 1 — Assess . . . 11

Stage 2 — Design . . . 12

Stage 3 — Generate/Build . . . 12

Stage 4 — Validate (gates) . . . 13

Stage 5 — Expand . . . 13

Testing & Validation 14

T1. Scenario coverage validation . . . 14

T2. Metric correctness (fixture) validation . . . 14

T3. Gate enforcement validation . . . 14

T4. Privacy / no-PHI validation . . . 15
T5. Reproducibility validation . . . 15
T6. Regression comparison validation . . . 15
T7. End-to-end pilot validation . . . 15
Troubleshooting 16
Scenarios error out or are skipped (coverage gate FAILS) . . . 16
Adapter import/resolve fails . . . 16
A metric looks wrong even though the model seems fine . . . 16
Gate reports PASS when it should FAIL (or vice versa) . . . 17
Privacy gate FAILS / boundary not attested . . . 17
Reproducibility: two runs of the same model disagree . . . 17
Regression check reports no regression on an obviously worse model . . . 17
`harness_config.py validate` reports a secret in the config . . . 18
Vendor model framework behaves differently than documented . . . 18
Best Practices 18
Treat scenarios, metrics, and gates as versioned data — not code . . . 18
Set gate thresholds before you see results . . . 18
Keep the model behind the adapter — never leak vendor SDKs into the harness . . . 18
Never write real PHI into the shared harness . . . 18
Cover the edges and the safety cases, not just the happy path . . . 19
Make every run reproducible . . . 19
Baseline deliberately, and re-baseline only on an approved improvement . . . 19
Keep secrets in the environment, never in assets . . . 19
Automate the gate into pre-production . . . 19
Retain failed runs . . . 19
Deliverables Summary 19
Documentation . . . 19
The 5 Python tools . . . 20
Appendix A — PowerShell Scripts 21

Executive Summary

Outcome

The organization gains a reusable **AI model testing and scenario-based evaluation harness** that proves an AI or machine-learning model is **safe and useful before it reaches production**. Every candidate model is evaluated the same way — against a defined library of **scenarios**, scored on agreed **metrics**, and passed or failed by explicit **pass/fail gates** — producing a defensible release decision instead of a one-off demo. The scenario suite, metrics, gates, and model adapter are versioned assets the organization keeps and re-runs for every model and every version.

Approach

The harness is built **pilot-first** on the house model — **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**. A model is registered behind a single abstract **Model Under Test adapter**, so the same evaluation works across any model or vendor framework without rewrites. The harness runs the scenarios on **synthetic or hospital-twin test data**, records results, and enforces coverage, quality, and privacy gates — a failing gate blocks promotion and triggers rollback. A baseline comparison then protects against silent quality regressions on the next candidate. Five Python tools cover readiness assessment, scenario execution, metrics-and-gate reporting, regression checking, and configuration.

Why it is safe

Evaluation runs only in an **isolated non-production environment** on **synthetic/twin data** — the harness writes **no real PHI** to shared locations, and the privacy gate and tests enforce that boundary. Vendor model frameworks and synthetic-data platforms are integrated **at the pattern level**; version-specific vendor internals are cited to their documentation or flagged to verify, never invented. Regulatory and legal determinations — de-identification acceptance, IRB/privacy sign-off, disclosure-risk acceptance — are **deferred to the organization's privacy, compliance, and legal functions**. This solution delivers the technical method and the evidence; the organization keeps a repeatable pre-production gate for AI it can trust.

Solution Overview

What this is

This solution builds a reusable **AI model testing and scenario-based evaluation harness** that lets a data/platform engineering team prove an AI or machine-learning model is **safe and useful before it reaches production**. Instead of one-off demos, the model is evaluated against a **defined library of scenarios**, scored on **explicit metrics**, and passed or failed by **pass/fail gates** — every run, the same way, on **synthetic or hospital-twin test data** that contains **no real Protected Health Information (PHI)**.

The harness is model-agnostic. Any model is plugged in behind a small **abstract adapter interface** (a "model under test" contract), so the same scenarios, metrics, and gates work across a local model, a hosted model, a vendor model framework, or a future replacement — without rewriting the evaluation. This solution defines that pattern; it does **not** ship or wrap any specific vendor SDK.

The three artifacts every run centers on are:

• **Evaluation Scenario Suite – Pilot – v1** — a versioned set of test scenarios (inputs + expected behavior / reference answers) drawn from the AI use cases in scope.

• **Metric Definition Set – Pilot – v1** — the metrics computed per scenario (e.g. exact/normalized match, task-appropriate score, latency, refusal/safety flags) and how each is aggregated.

• **Evaluation Gate Policy – Pilot – v1** — the thresholds that turn metrics into a **PASS/FAIL** decision for the whole run (and per scenario group), plus the rollback rule when a gate fails.

How it works

1. A model is registered as a **Model Under Test** behind the adapter interface — the harness calls a single, well-defined method (`predict(...)` over a scenario input) and never depends on a vendor's payload shape.

1. The harness loads the **Evaluation Scenario Suite** and the **Metric Definition Set**, runs each scenario against the model on **synthetic/twin data**, and records raw outputs and per-scenario metrics into a **Run Record** (`Evaluation Run – <pilot> – <timestamp>`).

1. The **Metrics Reporter** aggregates the Run Record and applies the **Evaluation Gate Policy**, producing a **PASS/FAIL** result with the failing scenarios enumerated — the signal a release decision hangs on.

1. A **Regression Checker** compares a new Run Record against a stored **baseline** run so a model, prompt, or data change cannot silently degrade quality.

1. The whole thing is **reusable**: the scenario suite, metrics, gates, and adapter are versioned assets the organization keeps and re-runs for every model, every candidate version.

Because scenarios and gates are data (files), not code, the harness is **repeatable and comparable**: the same run on the same inputs yields the same decision, and two models can be judged on identical terms.

Target model / data

The evaluation runs against **synthetic or hospital-twin** data produced under the organization's synthetic-data controls. Where the AI use case is clinical, scenario inputs align to the target clinical model in scope — **FHIR**, **OMOP CDM**, or the source system's native schema — so the model is exercised on structurally realistic records. **No real PHI** is written into the shared harness; any reference statistics used for realism are read-only and minimized under the governing privacy controls. The *format* of a scenario input (a FHIR bundle excerpt, an OMOP row set, a de-identified note) is a configuration of the suite, not a fixed assumption of the harness.

Benefits

• **Ship with evidence, not vibes.** Every candidate model carries a PASS/FAIL run against realistic, repeatable scenarios before broader adoption.

• **Comparable across models and versions.** Identical scenarios, metrics, and gates make model A vs.

model B — and v1 vs. v2 — an apples-to-apples decision.

• **Safe by construction.** Evaluation happens in an isolated non-production environment on

synthetic/twin data; the harness writes no real PHI to shared locations.

• **Reusable asset the org keeps.** The scenario suite, metric set, gate policy, and adapter outlive

any single model and any single vendor.

• **Regression protection.** A baseline comparison catches silent quality drops from a model, prompt,

or data change.

Limitations

• The harness measures **what the scenarios and metrics express** — coverage is only as good as the

scenario suite; gaps in scenarios are gaps in assurance. Best practices below drive coverage.

• It evaluates **behavior on test data**, not real-world outcomes; a PASS is a pre-production gate, not

a clinical or production guarantee.

• **Model frameworks and synthetic-data platforms are integrated at the pattern level.** Vendor-specific

console steps, SDK signatures, and version behaviors are **cited to the vendor's documentation or flagged to verify against your version** — never invented here.

• **Regulatory and legal determinations are out of scope.** Whether the synthetic/de-identified data is

acceptable for a given use, and any IRB/privacy sign-off, is **deferred to the organization's privacy, compliance, and legal functions**. This solution provides the technical method and the evidence.

The deployment model

Every step follows the house model for synthetic / AI-test solutions, **pilot-first with explicit gates and rollback**:

• **Assess** — inventory the AI use cases, the model(s) under test, the available synthetic/twin data,

and the isolated environment; define the evaluation goals and the pilot scope.

• **Design** — author the Evaluation Scenario Suite, the Metric Definition Set, and the Evaluation Gate

Policy; define the Model Under Test adapter contract.

• **Generate/Build** — wire the model adapter and the synthetic/twin test data into the harness; make a

dry run produce a Run Record.

• **Validate** — run the full suite, capture metrics, enforce the gates, and (fidelity/coverage/privacy

gates passing) establish the baseline. A failed gate triggers **rollback** — the candidate is not promoted and the run record is retained for diagnosis.

- **Expand** — broaden scenario coverage, onboard more models/use cases behind the same adapter, and

schedule the harness as a repeatable pre-production gate.

Requirements

These are the **solution-specific** prerequisites for the AI Model Testing & Scenario-Based Evaluation Harness. They build on — and do **not** repeat — the six category-wide requirements in `global-requirements-synthetic.md` (a Python data/engineering environment, a target clinical data model, an isolated non-production environment, privacy/governance stakeholders and policy, AI/QA use cases + pilot scope, and read-only minimized reference access). Confirm those six first; the items below are what this harness additionally needs.

> *Reminder of the boundary: evaluation runs on **synthetic/twin data in an isolated environment** with **no real PHI** in the shared harness. Vendor model frameworks are integrated at the pattern level; regulatory/legal determinations are deferred to privacy/compliance/legal.*

R1. A model exposed through a callable "Model Under Test" contract

- **Description:** Each model to be evaluated must be reachable behind a single callable interface — a function/class that takes a scenario input and returns the model's output (and, ideally, latency and any safety/refusal flags). This is the **Model Under Test adapter** (`MUT Adapter – <model> – v1`).
- **Why required:** The harness stays model- and vendor-agnostic by depending only on this contract, never on a vendor SDK's payload shape. Without it, scenarios cannot be run uniformly.
- **How to verify:** Confirm you can call the model once from the isolated environment and get a deterministic-enough response for a known input (record the call latency).
- **How to implement if missing:** Write a thin adapter that wraps whatever the model exposes (local inference call, hosted endpoint, or a vendor framework's client — **see that vendor's documentation** for the exact client call) into the harness's `predict(scenario_input) -> output` shape. Keep any credentials in environment variables, never in the adapter code.

R2. Ground truth / expected behavior for each scenario

- **Description:** For every scenario, a definition of "correct enough" — an expected answer, an acceptable-answer set, a reference label, or a rubric/threshold the metric can score against.
- **Why required:** Metrics and gates are meaningless without a reference; a scenario with no expected behavior cannot pass or fail.
- **How to verify:** Confirm each scenario in the Evaluation Scenario Suite has an `expected` (or a documented metric that needs none, e.g. latency or a safety-flag check).
- **How to implement if missing:** Work with the AI/QA owners of the use case to define expected

behavior per scenario; where exact answers don't exist, define a scored rubric or a tolerance band.

R3. Agreed evaluation metrics and pass/fail thresholds

• **Description:** The specific metrics to compute (e.g. normalized match rate, task score, latency percentile, refusal/safety-flag rate) and the numeric thresholds that constitute a PASS — captured as the **Metric Definition Set** and **Evaluation Gate Policy**.

• **Why required:** The gate decision is only as credible as the agreed thresholds; thresholds set after seeing results are not a gate.

• **How to verify:** Confirm each metric has an owner, an aggregation rule, and a threshold agreed with the AI/QA and governance stakeholders **before** the first scored run.

• **How to implement if missing:** Facilitate a short threshold-setting session; record the agreed numbers in the Gate Policy file and version it.

R4. Synthetic/twin test data mapped to the scenarios

• **Description:** The synthetic or hospital-twin records that scenario inputs draw from, in the target model's shape (FHIR / OMOP / native) and sufficient in volume and variety to cover the use cases.

• **Why required:** Scenarios must run on realistic, safe inputs; thin or unrepresentative test data produces a green gate that means nothing.

• **How to verify:** Confirm the synthetic/twin dataset exists in the isolated environment, matches the target model, and contains the cohorts/edge cases the scenarios reference. Confirm it holds **no real PHI**.

• **How to implement if missing:** Generate/obtain the synthetic/twin data (a separate synthetic-data or twin solution) before authoring scenarios that depend on it.

R5. A run/results store and a baseline location

• **Description:** A location in the isolated environment to write **Run Records** (per-run raw outputs + metrics) and to keep the **approved baseline** run for regression comparison — a directory, an artifact store, or a small database.

• **Why required:** Comparability and regression protection require durable, versioned run history; the gate and the regression check read from it.

• **How to verify:** Confirm a writable results path exists and is access-controlled, and that a baseline slot is designated.

• **How to implement if missing:** Create the results directory/store and record its path in the harness config (the config helper writes this).

R6. Determinism / reproducibility controls for the run

• **Description:** The controls needed to make a run **repeatable** — fixed random seeds, a pinned model version identifier, a recorded scenario-suite version, and a fixed data snapshot reference.

• **Why required:** "Comparable across models and versions" only holds if a re-run of the same inputs yields the same decision; uncontrolled nondeterminism makes regression comparison noise.

• **How to verify:** Run the same suite twice against the same model version and confirm metrics match within the tolerance recorded in the Gate Policy.

• **How to implement if missing:** Pin the model version, set and record seeds, snapshot the data, and stamp every Run Record with the scenario-suite and metric-set versions.

Global Requirements — Synthetic Data & AI Test-Enablement Solutions

These requirements apply to **every** solution in the **Synthetic** category (synthetic clinical data, hospital-twin simulation, and safe AI test enablement). Verify each before starting a solution's steps.

> ***Safety, vendor & regulatory boundary (applies to every Synthetic solution).*** *These solutions generate and validate **synthetic** (artificial) data and run in **isolated non-production** test environments. **Never place real Protected Health Information (PHI) into a shared synthetic harness, twin, or AI test environment** — where real data is touched at all it is read-only, minimized, and handled under the governing privacy controls. **Synthetic-data and digital-twin tools** (e.g. Synthea, and commercial platforms) and **AI/model frameworks** are integrated at the pattern/architectural level; specific vendor console internals are **cited or deferred** to that vendor's documentation, never invented. **Regulatory and legal interpretation** (HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, disclosure risk acceptance, and any ITAR/NERC/NRC or contract constraints) is **deferred to the organization's privacy, compliance, and legal functions** — the solutions provide the technical method and evidence, not legal determinations.*

1. A Python data/engineering environment

• **Description:** A supported **Python** environment (e.g. 3.10+) with the standard data toolchain (data frames, statistics, and the synthetic-data / validation libraries the solution uses).

• **Why required:** The automation and reusable assets are authored in Python (generation, de-identification, fidelity/utility metrics, scenario runners).

• **How to verify:** Confirm Python + the required packages install and run in the target environment.

• **How to implement if missing:** Provision the environment and install dependencies before starting.

2. A target clinical data model / standard

• **Description:** A defined **clinical data model or interoperability standard** to generate against —

e.g. **FHIR**, **OMOP CDM**, or the source system's own schema — plus the fields/domains in scope (patients, encounters, conditions, medications, labs, vitals, notes, etc.).

• **Why required:** Realistic synthetic data and a credible hospital twin require a target schema and clinically plausible relationships; validation is measured against it.

• **How to verify:** Confirm the target model/standard and the in-scope domains with the AI/QA teams.

• **How to implement if missing:** Select and document the target model before generating.

3. An isolated non-production test environment

• **Description:** A **segregated, non-production** environment (compute + storage) for the synthetic harness, hospital twin, and AI test runs — separate from production and from real-PHI systems.

• **Why required:** Safe evaluation of AI solutions before broader adoption depends on isolation; it also bounds any residual disclosure risk.

• **How to verify:** Confirm the environment is isolated, access-controlled, and not connected to production PHI stores.

• **How to implement if missing:** Stand up / obtain the isolated environment before running.

4. Privacy, governance & compliance stakeholders and policy

• **Description:** The governing **privacy, data-governance, and compliance** policies and the named stakeholders (privacy officer, security, and — where real source data or de-identification claims are involved — IRB/legal) whose requirements the work must satisfy.

• **Why required:** Regulated healthcare data constraints, de-identification acceptance, and enterprise data governance are policy decisions the technical method must align to, not defaults.

• **How to verify:** Confirm the applicable policies, disclosure-risk threshold, and approvers.

• **How to implement if missing:** Identify the policies and stakeholders before generating or releasing test data.

5. AI/QA use cases and a pilot scope

• **Description:** The **AI use cases and QA scenarios** the synthetic data / twin must support, and a **pilot scope** to validate on before broader rollout.

• **Why required:** Fidelity, coverage, and scenario design are driven by the AI use cases; every solution follows an **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model proven on a pilot.

• **How to verify:** Confirm the target AI use cases, evaluation goals, and a named pilot.

• **How to implement if missing:** Elicit the use cases and define the pilot before building the harness.

6. Reference/real-data access for fidelity (read-only, minimized)

• **Description:** Where synthetic **fidelity/utility** must be measured against reality, **read-only, minimized** access to reference statistics or a governed real dataset (under the privacy controls in §4) — or agreed reference distributions where real data cannot be used.

• **Why required:** Fidelity and validation assets compare synthetic output to real-world distributions and relationships; without a reference the claims are unfounded.

• **How to verify:** Confirm the reference source and its access/governance terms.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) before validating.

*The deployment model across every Synthetic solution is the same: **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, with explicit pass/fail (fidelity/privacy) gates and a documented rollback. Automation and reusable assets are delivered as **Python** — never real PHI is written to a shared harness, twin, or AI test environment.*

Implementation

Follow these steps in order. They implement the house model — **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** — **pilot-first**, with explicit **pass/fail gates** and a **rollback** rule. Every artifact is named with the `<Purpose> – <Scope> – <Stage>` convention and that exact name is reused in Testing, Troubleshooting, and the scripts. All work happens in the **isolated non-production environment** on **synthetic/twin data** — never real PHI in the shared harness.

The five Python tools referenced below live in `scripts/`:

• `eval_scope_assess.py` — assessment/profiling of readiness and scope.

• `scenario_runner.py` — loads scenarios + metrics, runs the model adapter, writes a Run Record.

• `metrics_reporter.py` — aggregates a Run Record and enforces the Gate Policy (PASS/FAIL).

• `regression_check.py` — compares a Run Record to the approved baseline.

• `harness_config.py` — scaffolds/validates the harness config and asset skeletons.

All scripts are **safe-by-default** (dry-run where they would write; no real PHI to shared locations). Run any with `--help`.

Stage 1 — Assess

1. **Confirm the six global requirements** (`global-requirements-synthetic.md`) and the six solution-specific requirements (`02-requirements.md`) are met. Do not proceed with an unmet environment-isolation or privacy requirement.

1. **Profile readiness.** Run the assessment tool to produce the **Eval Readiness Report – Pilot – v1**:

```
python scripts/eval_scope_assess.py --scope pilot --out reports/
```

It records the model(s) under test, the target data model (FHIR/OMOP/native), the isolated environment, the AI use cases, and the results-store path, and flags any gap. **Resolve every flagged gap before Stage**

2.

1. **Fix the pilot scope.** With the AI/QA owners, name **one** pilot use case and the model version to evaluate. Record it as the pilot scope in the readiness report.

Stage 2 — Design

1. **Scaffold the harness assets.** Create the config and empty asset skeletons:

```
python scripts/harness_config.py init --scope pilot --out harness/
```

This writes `harness/config.pilot.json` (results path, baseline slot, seeds, versions) and skeletons for the three core assets. **Never** put credentials in the config — the model adapter reads those from environment variables.

1. **Author the Evaluation Scenario Suite – Pilot – v1.** For each scenario, define: an `id`, the scenario `input` (drawn from the synthetic/twin data in the target model's shape), the `expected` behavior/answer (per requirement **R2**), and the metric group it belongs to. Cover the happy path, known edge cases, and at least one safety/refusal scenario. Save as `harness/scenarios.pilot.v1.json`.

1. **Author the Metric Definition Set – Pilot – v1.** Define each metric (name, how it scores one scenario, how it aggregates across the suite) — e.g. `normalized_match`, `task_score`, `latency_ms_p95`, `safety_flag_rate`. Save as `harness/metrics.pilot.v1.json`.

1. **Author the Evaluation Gate Policy – Pilot – v1.** For each metric (and the overall run), set the **PASS threshold** agreed under requirement **R3**, the per-metric direction (higher/lower is better), and the **rollback rule** (any hard-gate failure → candidate not promoted). Save as `harness/gates.pilot.v1.json`.

1. **Define the Model Under Test adapter (`MUT Adapter – <model> – v1`).** Implement the single `predict(scenario_input) -> {output, latency_ms, safety_flags}` contract (requirement **R1**), wrapping whatever the model exposes. For a vendor model framework, wrap that vendor's documented client call — **see the vendor's documentation; verify the client signature against your version.** Keep secrets in env vars.

Stage 3 — Generate/Build

1. **Wire the data and adapter, then dry-run.** Point the config at the synthetic/twin dataset and the adapter, and produce a Run Record without gating:

```
python scripts/scenario_runner.py \
  --config harness/config.pilot.json \
  --scenarios harness/scenarios.pilot.v1.json \
  --metrics harness/metrics.pilot.v1.json \
  --adapter mut_adapter:predict \
  --dry-run
```

`--dry-run` (default) executes the scenarios and prints a run summary **without** writing the Run Record. Confirm every scenario resolves an input, calls the adapter, and computes its metrics.

1. **Produce the first Run Record.** Re-run with `--commit` to write **Evaluation Run – Pilot – `<timestamp>`** into the results store:

```
python scripts/scenario_runner.py --config harness/config.pilot.json \
  --scenarios harness/scenarios.pilot.v1.json --metrics harness/metrics.pilot.v1.json \
  --adapter mut_adapter:predict --commit
```

Stage 4 — Validate (gates)

1. **Enforce the gates.** Aggregate the Run Record and apply the Gate Policy:

```
python scripts/metrics_reporter.py \
  --run reports/runs/<run-id>.json \
  --gates harness/gates.pilot.v1.json \
  --out reports/
```

This writes the **Evaluation Gate Report – Pilot – v1** and exits **non-zero on FAIL**.

• **Scenario/coverage gate (PASS/FAIL):** every scenario in the suite ran and produced a metric —

no errored or skipped scenarios above the tolerance in the Gate Policy.

• **Quality gate (PASS/FAIL):** each metric meets its PASS threshold (e.g. `normalized_match` ≥

threshold, `latency_ms_p95` ≤ threshold, `safety_flag_rate` ≤ threshold).

• **Privacy gate (PASS/FAIL):** the run wrote **no real PHI** to shared locations — the harness

operated only on synthetic/twin inputs and the outputs were retained under the isolated environment's controls. (Any disclosure-risk *determination* is deferred to privacy/compliance.)

1. **Gate decision & rollback.** If **all gates PASS**, approve the run as the **Evaluation Baseline –

Pilot – v1 **(next step). If** any gate FAILS**,** do not promote the candidate** — retain the Run Record and Gate Report for diagnosis (see `06-troubleshooting.md`), fix the model/scenario/data, and re-run from step 10. A failed gate is the rollback trigger.

1. **Establish the baseline.** Store the approved run as the baseline the regression check compares to:

```
python scripts/harness_config.py set-baseline \
  --config harness/config.pilot.json --run reports/runs/<run-id>.json
```

1. **Confirm regression protection works.** Run the regression check against the same run (expect zero

regressions) to prove the wiring:

```
python scripts/regression_check.py \
  --config harness/config.pilot.json --run reports/runs/<run-id>.json
```

Stage 5 — Expand

1. **Broaden coverage.** Add scenarios for more cohorts, edge cases, and safety conditions; bump the

suite to `Evaluation Scenario Suite – Pilot – v2` and re-baseline once it passes.

1. **Onboard more models/use cases.** Register additional models behind their own

`MUT Adapter – <model> – v1` and run them through the **same** scenarios, metrics, and gates for an apples-to-apples comparison.

1. **Make it a repeatable gate.** Schedule the run + report + regression check to execute for every

candidate model version before promotion, and keep the assets versioned so results stay comparable over time.

Testing & Validation

These procedures validate that the harness itself is correct and that a candidate model's run is trustworthy. Each has an **expected PASS/FAIL** signal. Run them in the isolated environment on synthetic/twin data. Artifact names match `03-implementation.md`.

T1. Scenario coverage validation

- **Goal:** Confirm every scenario in **Evaluation Scenario Suite – Pilot – v1** actually ran and produced a metric.
- **Procedure:** Run `scenario_runner.py` in `--dry-run` and inspect the summary; then check the Run Record's per-scenario status.
- **PASS:** Count of scenarios executed = count in the suite; zero errored/skipped scenarios above the Gate Policy tolerance.
- **FAIL:** Any scenario failed to resolve an input, call the adapter, or compute a metric — fix before scoring (see `06-troubleshooting.md`).

T2. Metric correctness (fixture) validation

- **Goal:** Confirm each metric in **Metric Definition Set – Pilot – v1** scores as intended.
- **Procedure:** Run the suite against a **fixture adapter** with known outputs (a stub that returns pre-set answers, including deliberate wrong ones), and check the metrics match hand-computed values.
- **PASS:** Every metric equals its expected value on the fixture within tolerance (e.g. a perfect fixture → `normalized_match = 1.0`; a half-wrong fixture → `0.5`).
- **FAIL:** Any metric disagrees with the hand-computed value — the metric definition is wrong, not the model.

T3. Gate enforcement validation

- **Goal:** Confirm the **Evaluation Gate Policy – Pilot – v1** turns metrics into the correct PASS/FAIL and that a FAIL blocks promotion.
- **Procedure:** Run `metrics_reporter.py` against (a) a Run Record known to clear all thresholds and (b) one known to breach a threshold.
- **PASS:** Case (a) exits **0** and reports PASS; case (b) exits **non-zero** and reports FAIL with the breaching metric/scenario enumerated.
- **FAIL:** A breaching run reports PASS, or the reporter exits 0 on a FAIL — the gate is not enforcing.

T4. Privacy / no-PHI validation

• **Goal:** Confirm the run wrote **no real PHI** to shared locations and operated only on synthetic/twin data.

• **Procedure:** Run `privacy_check.py`-equivalent step (the privacy gate in `metrics_reporter.py` plus a manual review): verify inputs came from the synthetic/twin dataset, outputs were retained only in the isolated results store, and no configured path points outside the isolated environment.

• **PASS:** All scenario inputs trace to the synthetic/twin dataset; no output path resolves outside the isolated environment; the privacy gate reports PASS.

• **FAIL:** Any input traces to a real-PHI source, or any write target is outside the isolated environment — **stop**; this violates the safety boundary. (Note: the *disclosure-risk determination* itself is deferred to privacy/compliance/legal — this test checks the technical control, not the legal acceptance.)

T5. Reproducibility validation

• **Goal:** Confirm a re-run of the same suite against the same model version yields the same decision.

• **Procedure:** With seeds fixed and the model version pinned (requirement **R6**), run the suite twice and diff the aggregated metrics.

• **PASS:** The two runs' metrics match within the reproducibility tolerance recorded in the Gate Policy, and both yield the same PASS/FAIL.

• **FAIL:** Metrics differ beyond tolerance — nondeterminism is uncontrolled; pin/seed before trusting regression comparisons.

T6. Regression comparison validation

• **Goal:** Confirm `regression_check.py` detects a real quality drop against the **Evaluation Baseline – Pilot – v1**.

• **Procedure:** Compare (a) the baseline run to itself (expect zero regressions) and (b) a deliberately degraded run (e.g. fixture adapter with more wrong answers) to the baseline.

• **PASS:** Case (a) reports no regression and exits 0; case (b) reports the regressed metric(s) and exits non-zero.

• **FAIL:** A degraded run reports no regression — the check is not comparing correctly.

T7. End-to-end pilot validation

• **Goal:** Confirm the full Assess→Validate flow produces a defensible gate decision on the real pilot model.

• **Procedure:** Execute steps 9–14 of `03-implementation.md` against the pilot model and dataset.

• **PASS:** A committed **Evaluation Run – Pilot –** `<timestamp>`, an **Evaluation Gate Report – Pilot – v1 **with an explicit PASS/FAIL, and (on PASS) an established** Evaluation Baseline – Pilot – v1**.

• **FAIL:** Any stage cannot produce its artifact — resolve before declaring the harness ready for Expand.

Troubleshooting

Common failure modes when standing up and running the AI Model Testing & Scenario-Based Evaluation Harness, with resolutions. Artifact names match `03-implementation.md`. All work stays in the isolated environment on synthetic/twin data — never real PHI in the shared harness.

Scenarios error out or are skipped (coverage gate FAILS)

• **Symptom:** `scenario_runner.py` reports `errored > 0`, or `metrics_reporter.py` fails the coverage gate.

• **Likely causes:** a scenario `input` doesn't resolve against the synthetic/twin dataset; the adapter raised on a malformed input; the scenario shape doesn't match what the adapter expects.

• **Resolution:** inspect the Run Record's `per_scenario[].error`. Fix the scenario input or the adapter's input handling in **Evaluation Scenario Suite – Pilot – v1**; re-run `--dry-run` until `errored = 0`. Do not lower `coverage.max_errored` to hide a broken scenario.

Adapter import/resolve fails

• **Symptom:** `adapter target 'module:callable' is not callable` or an `ImportError`.

• **Likely causes:** the `--adapter module:function` path is wrong, the module isn't on `PYTHONPATH`, or the adapter isn't a callable returning the `{output, latency_ms, safety_flags}` contract.

• **Resolution:** confirm the module is importable from the run directory; confirm the target is a function or `Class.method`; validate the return shape. Use `--adapter fixture` to prove the harness itself works, then point back at the real **MUT Adapter –** `<model>` **– v1**.

A metric looks wrong even though the model seems fine

• **Symptom:** `normalized_match` is 0.0 for answers that look correct.

• **Likely causes:** whitespace/case/punctuation differences; `expected` doesn't match the model's output format; the wrong metric for the task (exact match where `contains` or a rubric was meant).

• **Resolution:** run test **T2** (fixture) to confirm the metric scores as designed; normalize the

`expected` values or switch the metric in **Metric Definition Set – Pilot – v1**. Remember metrics are only in `PER_SCENARIO_METRICS` if implemented in `scenario_runner.py`.

Gate reports PASS when it should FAIL (or vice versa)

- **Symptom:** a run that breaches a threshold still shows OVERALL PASS.
- **Likely causes:** wrong `direction` in the Gate Policy (`min` vs `max`), a threshold set too loose, or the metric name in `gates.quality[].metric` doesn't match the aggregate key.
- **Resolution:** verify each quality gate's `metric` name matches a key in the run's `aggregates`, and that `direction` matches the metric's meaning (`latency_ms_p95` and `safety_flag_rate` are `max`; match/score metrics are `min`). Re-run test **T3** with a known-bad run.

Privacy gate FAILS / boundary not attested

- **Symptom:** `metrics_reporter.py` or `regression_check.py` fails the privacy/boundary check.
- **Likely causes:** the run wasn't produced by this harness on synthetic/twin data, a `real_phi_present` flag is set, or a configured path points outside the isolated environment.
- **Resolution: stop and investigate** — confirm all scenario inputs trace to the synthetic/twin dataset and every write target is inside the isolated environment (test **T4**). Never suppress this gate. The disclosure-risk *determination* remains with privacy/compliance/legal; this check only enforces the technical control.

Reproducibility: two runs of the same model disagree

- **Symptom:** identical suite + model version yields different metrics across runs (test **T5** FAILS).
- **Likely causes:** unseeded randomness in the model or sampling, an unpinned model version, or a shifting data snapshot.
- **Resolution:** pin the model version in `config.*.json` (`model_version`), set seeds, and snapshot the data (requirement **R6**). Record all three in the Run Record. Only trust regression comparisons once T5 passes.

Regression check reports no regression on an obviously worse model

- **Symptom:** a degraded candidate passes `regression_check.py`.
- **Likely causes:** tolerance set too high, wrong `metric_directions`, or the baseline pointer is stale/wrong.
- **Resolution:** confirm `regression_tolerance` is tight, verify `metric_directions` (higher-better vs lower-better), and confirm `baseline_run` points at the intended **Evaluation Baseline – Pilot – v1**. Re-run test **T6** with a deliberately degraded fixture run.

`harness_config.py validate` reports a secret in the config

- **Symptom:** validation FAILS with "config appears to contain a secret-like key".
- **Likely cause:** credentials or endpoints with tokens were placed in `config.*.json`.
- **Resolution:** remove them. Credentials belong only in environment variables read by the adapter;

config files are versioned artifacts and must never hold secrets.

Vendor model framework behaves differently than documented

- **Symptom:** the adapter's wrapped vendor client call returns an unexpected shape or errors.
- **Likely cause:** version drift — the framework's client signature or response changed across versions.
- **Resolution:** this solution integrates model frameworks **at the pattern level** only. Consult the

vendor's documentation and **verify the client signature/response against your installed version**; adjust the adapter accordingly. Do not assume internals — keep the harness depending only on the `predict(...)` contract.

Best Practices

Durable practices for running the AI Model Testing & Scenario-Based Evaluation Harness well. They keep evaluations **safe, repeatable, and comparable** over time.

Treat scenarios, metrics, and gates as versioned data — not code

Keep the **Evaluation Scenario Suite**, **Metric Definition Set**, and **Evaluation Gate Policy** as versioned files (…`pilot.v1.json`, …`v2.json`). Bump the version on any change and stamp every Run Record with the versions it used. This is what makes two runs comparable and a regression check meaningful.

Set gate thresholds before you see results

Agree PASS thresholds with the AI/QA and governance owners **before** the first scored run (requirement **R3**). A threshold chosen after seeing the numbers is not a gate — it is a rationalization. Record the agreed numbers in the Gate Policy and change them only through a deliberate, versioned decision.

Keep the model behind the adapter — never leak vendor SDKs into the harness

All model access goes through the single `MUT Adapter` `predict(...)` contract. Never scatter vendor client calls through the runner, metrics, or gates. This keeps the harness model- and vendor-agnostic, lets you compare models on identical terms, and makes swapping a model a one-file change. For any vendor framework, wrap its **documented** client call and **verify the signature against your version** — do not assume.

Never write real PHI into the shared harness

Evaluation runs on **synthetic/twin data** in the **isolated environment**. Any reference statistics used for realism are read-only and minimized under the governing privacy controls. Keep every write target inside the isolated environment; the privacy gate and test **T4** enforce this. Leave the **disclosure-risk and de-identification determinations to privacy/compliance/legal** — the harness supplies the technical evidence, not the legal sign-off.

Cover the edges and the safety cases, not just the happy path

Coverage is the ceiling on assurance. Deliberately include edge cohorts, malformed/boundary inputs, and **safety/refusal** scenarios. Grow the suite as new failure modes are discovered — every production surprise should become a scenario so it can never regress silently.

Make every run reproducible

Pin the model version, fix seeds, snapshot the data, and record all of it in the Run Record (requirement **R6**). Reproducibility is the precondition for trusting a regression comparison; without it, diffs are noise.

Baseline deliberately, and re-baseline only on an approved improvement

Promote a run to **Evaluation Baseline** only after all gates PASS. Re-baseline when a model or suite genuinely improves — not to make a red regression go green. The baseline is the contract the next candidate must beat.

Keep secrets in the environment, never in assets

Model credentials and endpoints live in environment variables read by the adapter — never in `config.*.json`, scenarios, or scripts. Config files are versioned artifacts; secrets are not.

Automate the gate into pre-production

Once stable, run the suite + report + regression check automatically for every candidate model version before promotion. A harness that only runs when someone remembers is not a gate.

Retain failed runs

Keep failing Run Records and Gate Reports — they are the diagnosis trail and the raw material for new scenarios. Do not delete a red run just because the candidate was rejected.

Deliverables Summary

What the buyer receives — a complete, reusable **AI model testing and scenario-based evaluation harness** for safe pre-production evaluation of AI/ML models on synthetic/twin data.

Documentation

- **Solution Overview** — the evaluation model: scenarios, metrics, synthetic/twin test data,

pass/fail gates, the Model Under Test adapter pattern, and the Assess→Design→Generate/Build→Validate→Expand deployment model.

- **Requirements Guide** — the six solution-specific prerequisites (model-under-test contract, ground truth, agreed metrics/thresholds, synthetic/twin test data, run/baseline store, reproducibility controls), each with how to verify and how to implement — on top of the six category-wide requirements.
- **Implementation Guide** — exact, ordered, pilot-first steps to author the scenario suite, metric set, and gate policy; wire the model adapter and synthetic/twin data; run evaluations; enforce the pass/fail gates; baseline; and expand — with the rollback rule.
- **Testing & Validation** — seven procedures (coverage, metric correctness, gate enforcement, privacy/no-PHI, reproducibility, regression, end-to-end pilot), each with an explicit PASS/FAIL.
- **Best Practices** — durable practices for safe, repeatable, comparable evaluation.
- **Troubleshooting** — common failure modes and resolutions.
- **Executive Summary** — the outcome, approach, and why it is safe.

The 5 Python tools

1. `eval_scope_assess.py` — *Assessment / profiling.* Profiles readiness and scope (models under test, target data model, isolated environment, use cases, results store) and writes an **Eval Readiness Report**, flagging every gap. Read-only.

1. `scenario_runner.py` — *Scenario execution.* Loads the Evaluation Scenario Suite and Metric Definition Set, calls each model through the abstract **Model Under Test adapter**, computes per-scenario metrics, and writes a **Run Record**. **Dry-run by default** (`--commit` to persist).

1. `metrics_reporter.py` — *Fidelity/coverage report + gate enforcement.* Aggregates a Run Record and applies the **Evaluation Gate Policy** (coverage, quality, and privacy gates), writes the **Evaluation Gate Report**, and **exits non-zero on FAIL** so it can gate a pipeline.

1. `regression_check.py` — *Privacy/validation & regression check.* Compares a new Run Record to the approved **Evaluation Baseline**, flags any metric regression beyond tolerance, and verifies the run stayed within the isolated (no-real-PHI) boundary. Exits non-zero on a detected regression.

1. `harness_config.py` — *Scope/config helper.* Scaffolds and validates the harness config and the three core asset skeletons (scenarios, metrics, gates), and manages the baseline pointer (`set-baseline`). Safe: `init`/`validate` don't overwrite without `--force`.

All tools are **Python 3.10+**, standard-library-first, **safe-by-default** (no real PHI written to shared locations; dry-run/opt-in for anything that writes), and carry no hardcoded secrets. Run any with `--help`.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

eval_scope_assess.py

harness_config.py

metrics_reporter.py

regression_check.py

scenario_runner.py